DBOps Operator Guide

Daily operations, backup, recovery, and safe administration

Purpose and operating model

DBOps is a self-hosted control plane for registering database targets, monitoring health, operating PostgreSQL snapshots, and managing remote database hosts. Select and verify the target before any action. Dashboard monitoring is separate from changes made directly in an external database.

1. Sign in and orient yourself

Sign in with your assigned account. If MFA is enabled, complete the Microsoft Authenticator step. At first sign-in after a password reset, enter the temporary password as Current password and choose a different new password of at least 12 characters.

Console manages targets and servers; Overview shows health and performance; Backups handles snapshots; Restore Center and Restored DBs handle recovery; Data Browser is read-only; Activity records recent operator actions.

The active-target selector changes the database used by monitoring and relevant actions. Confirm its name before pinging, browsing, taking a snapshot, or starting a restore. Backup schedules select their own source database.

Refresh reloads metrics. Live refresh can be paused. The theme toggle changes appearance. Ctrl+K (Command+K on Mac) opens the command palette.

2. Register and manage database targets

Choose Add database instance in Console and enter a clear display name, engine, and connection details. Registration stores a connection record; it does not copy, create, or migrate the database. Test connectivity before relying on monitoring. Use the narrowest database permissions practical.

Supported engines are PostgreSQL, MongoDB, MySQL/MariaDB, and Redis. Metric coverage differs by engine and permissions. Snapshots, archive verification, and restore currently support PostgreSQL only.

Changing a display name changes only its DBOps label. Unregistering removes only the app record. Treat full connection URLs as credentials.

3. Read the dashboard and inspect data

Overview presents connectivity and available signals such as connections, database size, cache ratios, slow queries, deadlocks, and recognized migration history. Missing data can reflect engine limits, permissions, or connectivity. A green dashboard is not a substitute for database monitoring.

Migration History reads recognized PostgreSQL migration ledger tables. It reports records; it never applies or reverses migrations.

Data Browser performs read-only inspection and returns at most 200 rows per request.

Ping checks connectivity. Kill cancels a running query and can disrupt an application. Confirm the target and query before using it.

Backup and recovery workflows

4. Create and verify PostgreSQL snapshots

Open Backups and choose Take Snapshot. Confirm the PostgreSQL target, archive format, compression, optional encryption, and description. Manual snapshots are stored locally. Select remote servers for additional copies. Wait for completion and review copy-verification details.

Custom archive is recommended for flexible pg_restore recovery. Plain SQL is useful for script-oriented workflows. Encrypted archives require the configured key; losing or changing it can make them unreadable.

Verify checks archive integrity but does not test application recovery. Download retrieves a local archive. Deleting the local snapshot removes only its local archive and entry; remote copies remain.

5. Configure an automated schedule

Open Backup Schedule, choose the PostgreSQL source, select daily, weekly, or monthly, enter Sri Lanka local time, and set Active. The editor converts the time to UTC and shows both zones for the next run. Default: daily at 07:30 Sri Lanka time (02:00 UTC). Weekly schedules use the selected Sri Lankan weekday. Monthly schedules allow days 128. Every scheduled snapshot is first stored locally. Selected mounted repositories and remote servers receive checksum-verified copies. The schedule source is independent of the active dashboard target.

After saving, review source, status, destinations, and next run. Keep a loaded custom schedule only if it remains intended. Disabled pauses future runs.

6. Restore safely

In Restore Center, select a completed snapshot and inspect source, timestamp, format, and available local or verified remote copy. Choose a registered PostgreSQL destination. Prefer a new, unused database name to protect the original. Remote copies support new-database restores only.

A new-database restore requires the exact phrase RESTORE. Writing into an existing database requires RESTORE EXISTING and writes snapshot objects and data into that database. This can disrupt workloads or replace data; verify source and destination before confirming.

Follow job status and logs to completion. Snapshot exports omit owner and privilege settings; configure roles, grants, and ownership separately when required.

Restored DBs lists clone restores. Add to Targets registers one for monitoring. Remove from Targets unregisters it but does not delete its database.

7. Recovery readiness

Check that scheduled snapshots complete, remote copies show checksum verification, and storage capacity is sufficient. Default local retention is 14 days. Back up encryption keys separately from encrypted archives. Periodically restore into a disposable database and confirm the application can use it.

Remote infrastructure, administration, and response

8. Register servers and provision containers

Register a remote Linux Docker server with a display name, SSH-reachable host, port, account, and permitted database/snapshot paths. Inspect its SSH fingerprint, compare it with a trusted provider or server source, and confirm verification. Use absolute Linux paths whose parent directories already exist. SSH credentials are encrypted in the app registry.

Server health and managed-container panels report connectivity, resources, and DBOps-created containers. Renaming changes only the app label. Removing registration does not stop containers or delete files.

Provisioning requests engine, DBOps label, unique container name, database name/user (except Redis), password of 16128 characters, supported image, and host port. Review the connection URL and destination before creating.

The app checks that the host port is free, then checks again before creation. This is a point-in-time check; Docker final bind is authoritative. New containers and data directories are created under configured paths; existing containers are not changed.

9. Storage, notifications, and alerts

Mounted storage targets provide repository destinations. Registered remote servers provide SSH copy destinations. Confirm copies are completed and checksum-verified before relying on them for recovery.

Alerts can use thresholds for connection saturation, cache efficiency, slow queries, and migration-ledger changes. Telegram, email, and webhook channels may be configured; test delivery and protect tokens and recipient details. Alerts are investigation signals, not proof of impact or a completed fix.

10. Users and access

Administrators create accounts with the correct role and optional MFA. If MFA is enabled, securely deliver the individual QR code or setup key. Password resets sign the user out and require a new password at next sign-in; MFA remains enabled. Admin has additional account-management authority. Avoid shared accounts.

11. Operational safety and troubleshooting

Connection failure: verify active target, host and port, credentials, network route, firewall, and database permissions; then test again.

SSH or provisioning failure: verify host reachability and fingerprint, SSH/Docker access, absolute paths, supported image, unique container name, password limits, and port. Verify host-key changes with a trusted source; never bypass host-key checking.

Snapshot or restore failure: record the job ID and redacted error, check disk space and destination permissions, confirm encryption key, and ensure a new database name is unused. Do not retry an existing-database restore until its impact is understood.

Use Activity to correlate actions and timestamps. Never share passwords, API keys, full connection URLs, MFA setup codes, private keys, or unredacted logs.

12. What DBOps does not do

DBOps is an operations interface, not a managed database service. Registration does not guarantee credentials are read-only. Migration history does not roll migrations back. Ping does not certify health, and archive verification does not prove application recovery. Removing a registration does not delete underlying data. Maintain independent access controls, monitoring, backups, and tested recovery procedures.

Operator closeout

Before ending a maintenance window, record the target and operator, confirm snapshot and copy status, capture restore outcomes, review alerts and Activity, and communicate unresolved failures through the approved incident process.